

3. Bölüm

Derleyiciler ve Çalıştırma Ortamı

3.1 Derleyici ve .NET Framework

Genel amaçlı bilgisayarların çok kullanılmasıyla birlikte metin editörleri de (Windows Not Defteri, Mac TextEdit, Notepad++, Epsilon, EMACS, nano, vim veya vi) işletim sistemlerinin bir parçası olmuştur. Metin editörlerinde yazılan programlar ise **derleyiciler (compiler)** tarafından makine koduna çevrilerek **icra edilebilir (executable)** dosya oluşur. Bu dosya, işletim sisteminin yardımıyla işlemci tarafından çalıştırılır.

C# dilinde programlama öğrenmeye başlamak için;

1. İlk adım, programı yazabileceğiniz bir metin editörü kullanmak ve yazdığınız programa ilişkin kaynak kodları *.cs uzantısı ile dosya olarak saklamaktır. Tüm kaynak kodlar doğal olarak insan tarafından okunabilen metin dosyasıdır.
2. İkinci adım ise **.NET Yazılım Çerçevesinin (.NET Framework, kısaca .NET)** uygun sürümünü işletim sisteminizde **(operating system)** kurmaktır.
3. Üçüncü adım ise yazdığınız kodu derleyerek çalışabilen bir icra edilebilir dosyayı oluşturup çalıştırmaktır. C# derleyicisi, kaynak dosya içerisinde yazılı C# talimatlarını **(statement)** .NET'in tarafından icra edilecek bir ara dosyaya dönüştürür.

C# dilinde derleme sonrasında .NET platformu tarafından çalıştırılacak **ara kod** elde edilir. .NET platformu, her işletim sistemi için ayrı kurulur. Amacı derlenen bu ara kodu, **işletim sistemi ve işlemciden bağımsız olarak her yerde çalıştırmaktır!**

.NET platformu, çalışan uygulamalarına çeşitli hizmetler sağlayan bir **çalışma ortamıdır (run-time)**. İki ana bileşenden oluşur:

- ♦ **Temel Sınıf Kütüphanesi BCL (Base Class Library)**: Yazılımcıların kendi uygulamalarını geliştirirken esas alacağı, çağırabileceği, test edilmiş ve yeniden kullanılabilir bir kütüphanedir. Bir başka adı **.NET sınıf kütüphanesidir (.NET Class Library)**.
- ♦ **Ortak Dil Çalışma Ortamı CLR (Common Language Runtime)**: .NET için derlenmiş uygulamaları çalıştıran ve yöneten motordur **(engine)**. İş parçacığı **(thread)** yönetimi, çöp toplama GC **(garbage collector)**, veri tipi güvenliği **(type safety)**, istisna yönetimi **(exception handling)** ve daha fazlasını içeren hizmetler sağlar.

Bu bileşenler, çalışan uygulamalar için aşağıdaki hizmetleri sunar:

- ♦ **Bellek yönetimi**: Çalışma ortamı (CLR), birçok programlama dilinde, programcılar bellek ayırmayı ve serbest bırakmayı ve nesne ömrünü işlemekten sorumludur.
- ♦ **Ortak tip sistemi CTS (Common Type System)**: Geleneksel programlama dillerinde temel tipler derleyici tarafından tanımlanır, bu da başka dillerde yazılan uygulamaların ortak çalışabilirliği karmaşıktır. .NET, temel tipleri ortak tip sistemi tarafından tanımlanır ve .NET ile çalışan tüm dillerde ortaktır.
- ♦ **Kapsamlı bir sınıf kütüphanesi**: Programcılar, sık kullanılan alt düzey programlama işlemlerini işlemek için çok miktarda kod yazmak yerine, Temel sınıf kütüphanesinden (BCL) veri tipi ve üyeleri olan kolay erişilebilir bir kitaplık kullanır.
- ♦ **Yazılım geliştirme çerçeveleri**: .NET, web uygulamaları için ASP.NET, veri erişimi için ADO.NET, hizmet odaklı uygulamalar için WCF **(Windows Communication Foundation)** gibi yazılım geliştirme çerçevelerine sahiptir.
- ♦ **Birlikte çalışabilirlik (Interoperability)**: .NET altyapısı için uygulama geliştirmede kullanılan olan tüm diller (C#, VB.NET, F#, ...) derlendikten sonra ortak ara dile olan CIL **(Common Intermediate Language)**/MSIL **(Microsoft Intermediate Language)** koduna çalışma ortamı tarafından dönüştürülür. Bu özellik ile, bir dilde yazılan yöntemler diğer diller tarafından erişilebilir olur. Böylece programcılar tercih ettiği dilde uygulama oluşturmaya odaklanırlar.
- ♦ **Sürüm uyumluluğu**: Nadir özel durumlar dışında belirli bir .NET sürümü kullanılarak geliştirilen uygulamalar, daha sonraki sürümde değişiklik yapılmadan çalışır.
- ♦ **Yan yana çalıştırma**: .NET, aynı bilgisayarda ortak dil çalışma ortamının birden çok sürümünün var

olmasına izin vererek sürüm çakışmalarını çözmeye yardımcı olur.

- ◊ **Çoklu sürüm desteği:** Geliştiriciler, standart bir .NET sürümü tarafından desteklenen, birden çok sınıf kütüphanesi oluşturabilir.

3.2 Ortak Dil Tarifnamesi

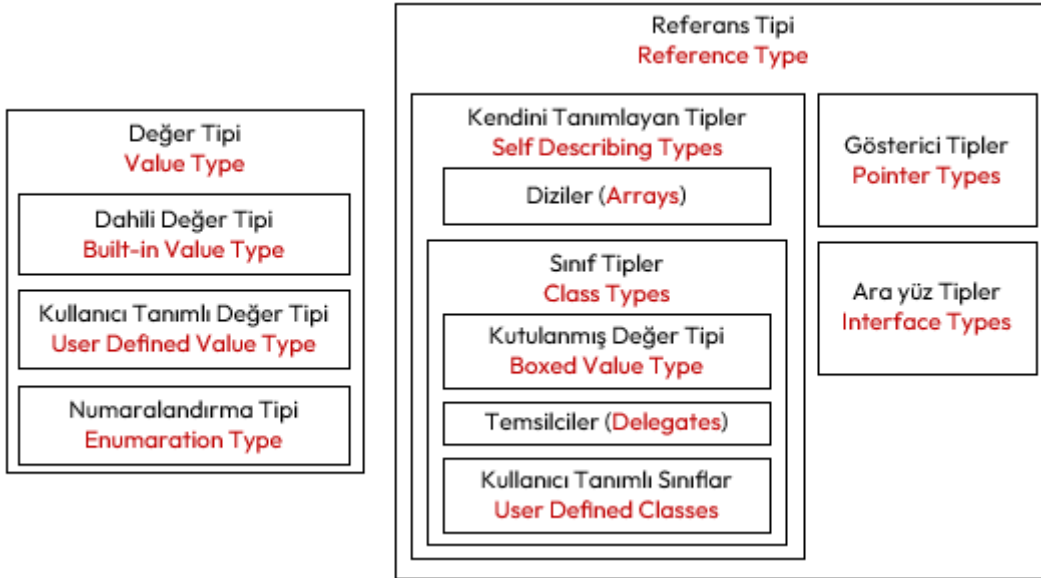
Ortak Dil Tarifnamesi CLS (**Common Language Specification**) programlama dillerinin nesne modellemesinden kaynaklanan farklılıklarını bertaraf etmek için geliştirilmiş standartlar dizisidir. Bunlar aşağıda belirtilen başlıklarda toplanmaktadır;

- ◊ Yapı (**structure**) standardı.
- ◊ Ortak ara dil (CIL) standardı, programlama dili ne olursa olsun, bu diller herkes tarafından bilinecek bir ara koda dönüştürülür ya da derlenir.
- ◊ Veri tipi standardı CTS (**Common Type System**) aşağıda anlatılmıştır.
- ◊ Olay (**event**) standartları.
- ◊ Verilerin bellekteki yerleşimine (**persistence**) ilişkin standartlardır. Aynı blok içinde tanımlanan **char**, **int**, **double** değişkenlerinin tanımlandığı örneği göz önüne alacak olursak; C dilinde, belleğe ilk önce **char**, sonra **int**, daha sonra **double** yerleşir (**row major**). Pascal dilinde ise ilk önce **double**, sonra **int**, son olarak da **char** yerleşir (**column major**).

İşte bu tip değişken yerleşimlerinden kaynaklanacak problemleri gidermek için bu standartlar oluşturulmuştur.

3.3 Ortak Tip Sistemi

Ortak Tip Sistemi (CTS), dillerdeki veri tipleri arasındaki farklılıkların bertaraf edilmesine yönelik standartlardır. Bu sistem ile bir dilde 4 bayt, diğer bir dilde 8 bayt olan gerçek sayı tanımlaması gibi karmaşıklıklar giderilmiştir.



Şekil 3.1: Ortak Tip Sistemi (CTS)

CTS, aşağıdaki veri tiplerini desteklemektedir:

1. Değer Tipleri (**value type**)
 - (a) Dâhili değer tipleri (**built-in value type**): **bool**, **int**, **byte**, **double**, **char** gibi veri tipleridir.
 - (b) Kullanıcı tanımlı değer tipleri (**user-defined value type**): **struct** gibi kullanıcı tarafından, yerleşik değer tiplerinin çeşitli şekillerde bir araya getirilip tek bir isim altında toplanmasıyla oluşan tiplerdir.
 - (c) Numaralandırma (**enumeration**): Bir kümeye ait elemanların her birine bir numara atanarak kod okunabilirliğini yükseltmek için tanımlanan **enum** tipleridir.
2. Referans Tipler (**reference type**)
 - (a) Gösterici tipler (**pointer type**)
 - (b) Arayüz tipler (**interface type**)
 - (c) Kendini tanımlayan tipler (**self-description type**)
 - (d) Diziler (**array**): Tek boyutlu, çok boyutlu ya da dizilerin dizisi (**jagged array**) gibi diziler.
 - (e) Sınıflar (**class**): **object** ve **string** gibi ön tanımlı sınıflar.

- (f) Kullanıcı tanımlı sınıflar (**user-defined class**)
- (g) Doğrudan ulaşımı engellenmiş ya da kutulanmış değer tipler (**boxed value types**)
- (h) Temsilciler (**delegate**): Yöntemleri referans gösteren tipler.

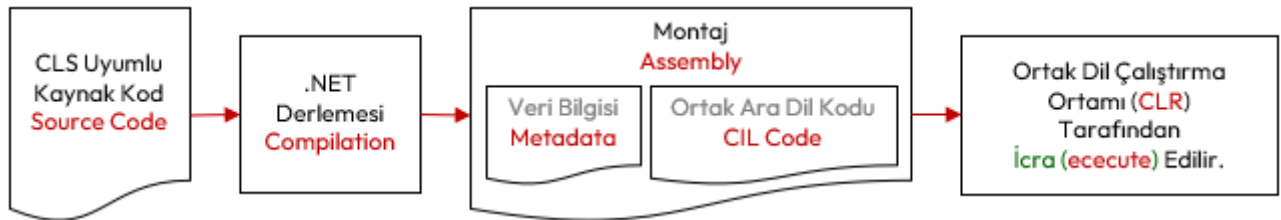
3.4 Derleme ve İcra Süreci

C ve C++ gibi klasik yazılım geliştirme araçlarıyla derlenen bir uygulama aşağıda gösterildiği gibi bir süreçten geçer. Yani kaynak kod derlenir ve doğrudan işletim sistemi tarafından işlemciye hedef gösterilerek çalıştırılacak şekilde işlemcinin emir kodlarından (**instruction code**) oluşan çalıştırılabilir kod (**executable code**) elde edilir (Şekil 3.2). Klasik derlemede sonucunda oluşan bu dosyaya, **ikili çalıştırılabilir doğal kod** (**binary executable native code**) veya kısaca **doğal kod** (**native code**) adı verilir.



Şekil 3.2: Klasik Derleme Süreci

.NET altyapısında ise kaynak kod derlenir ve çalıştırma ortamı (CLR) tarafından icra edilecek şekilde **ortak ara dil** (CIL) ve **veri bilgisinden** (**metadata**) oluşan bir dosya elde edilir. Derlenmiş bu .NET uygulamasına, **montaj** (**assembly**) adı verilir (Şekil 3.3). Bu isim, 1.9 başlığında anlatılan sembolik isimlerle yazılan kodlarla karıştırılmamalıdır.



Şekil 3.3: .NET Framework Derleme Süreci

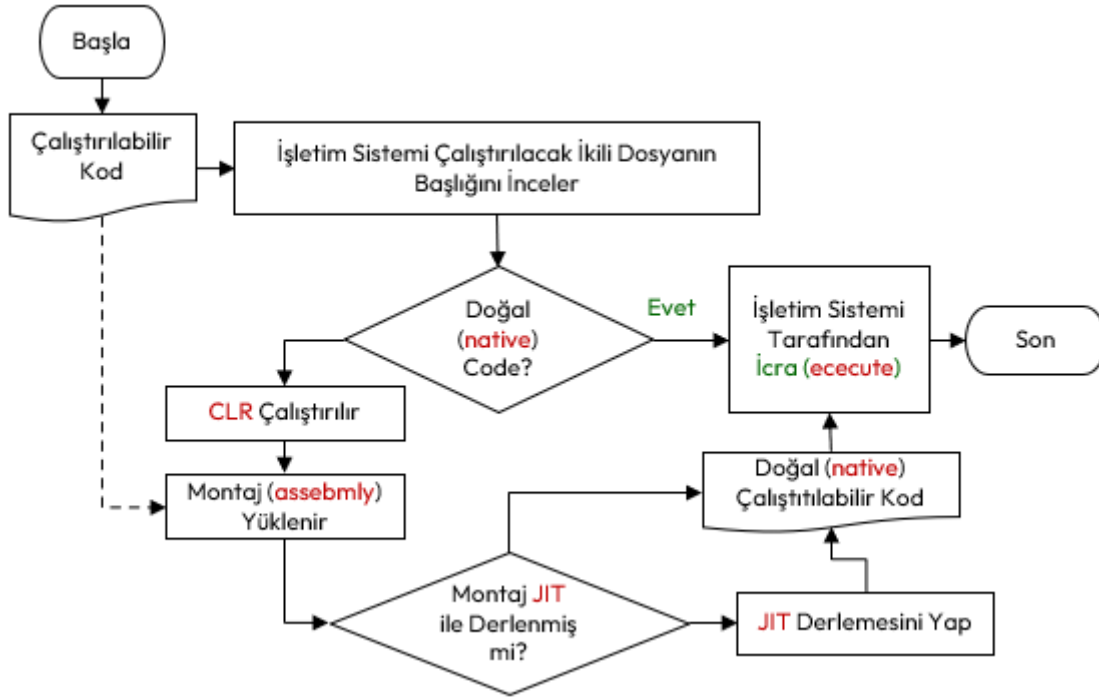
.NET altyapısında çalışacak kaynak kod, ortak dil tarfnamesinde (CLS) belirtilen standartlara uymak zorundadır. İçinde ara dil (CIL) kodu olan bu **montaj** (**assembly**) uygulama, işlemcinin anladığı emir kodları (**instruction code**) ile hiçbir ilgisi yoktur ve işletim sistemine bağımlı değildir. .NET platformu için yapılmış derleme işlemine, **yönetilmiş derleme** (**managed compilation**) adı verilir. Bunun yanında CLS uyumlu olarak hazırlanmış kaynak kodlara **yönetilmiş kod** (**managed code**) adı verilir. CLS uyumlu hazırlanmayan kodlara ise **yönetilmemiş kod** (**unmanaged code**) adı verilir. **Yönetilmiş kodlar CLR tarafından, yönetilmemiş kodlar ise işletim sistemi tarafından çalıştırılır.**

Peki işletim sistemi; Çalıştırılabilir (**executable**) uygulamayı işlemcinin mi yoksa .NET platformunun mu çalıştıracağını nasıl anlar? Windows işletim sisteminde çalıştırılabilir uygulamalar *.exe uzantısına sahiptir. Windows, derlenmiş bir çalıştırılabilir uygulamayı Şekil 3.4'deki gibi yapar. Klasik derleme sonucunda oluşan doğal kod (**native code**), işletim sistemi tarafından doğrudan belleğe yüklenerek çalıştırılır. .NET platformu için derlenen montaj (**assembly**) uygulamanın çalışması için, ortak dil çalıştırma ortamının (CLR) işletim sisteminde çalışıyor olması gerekir. Çalışıyor olan CLR, ilgili montaj uygulamasını belleğe yükler ve ani derleyiciler olan **JIT Compiler** (**Just-in-time Compiler**) tarafından derlenerek işlemcinin anlayacağı **doğal kod** (**native code**) elde edilir. Burada oluşan doğal kod, CLR kontrolünde, işletim sistemi tarafından çalıştırılır. İcra edilebilir bu doğal kod, her platformda farklıdır. Windows işletim sisteminde .exe, Linux işletim sisteminde ise uzantısız bir dosyadır.

Ortak dil çalışma ortamı (CLR), ortak dil altyapısı (CLI) standartlarına uyumlu olarak derlenmiş ve kurulu olduğu işletim sisteminin kaynaklarını kullanarak oluşturulan CIL kodunun doğru bir şekilde çalıştırılmasını sağlayan altyapıdır.

Microsoft tarafından Visual Basic, Visual Java, Visual C++ dillerinde küçük değişiklikler yapılarak CLS uyumlu hale getirilmiştir. Bu diller .NET dilleri olarak da adlandırılır. Böylece yazılımcı takımında dil farklılıklarından doğan geliştirme problemlerine de çözüm bulunmuştur. Böylece yazılımcıların .NET mimarisine geçmek için harcaacağı eğitim ve zamandan tasarruf sağlamıştır.

Ortak dil çalıştırma ortamının montaj uygulamayı belleğe yükleyip, ani derleyicilerden geçirerek işlem-



Şekil 3.4: Derlenmiş bir Çalıştırılabilir Uygulamanın Çalışma Süreci

cinin çalıştıracağı doğal kod (**native code**) elde edilir. Bazı durumlarda içinde bulunulan işletim sistemine uygun olarak montaj (**assembly**) uygulaması bu doğal koda önceden çevrilir. Buna **vaktinden önce doğal derleme** NAOT (**Native Ahead-Of-Time**) adı verilir. Bu durum için .NET derleyicileri optimize edilmemiştir. Bazı test senaryoları için kullanılabilir.

3.5 Ortak Ara Dil Kodu

Yapısal programlama ve fonksiyon kavramının bilgisayarlarda kullanılmasıyla birlikte işlemcilerde de değişiklikler olmuştur. Basit bir işlemci 1.5 başlığında anlatılan işlemciye yeni kaydediciler de eklenmiştir;

- ◊ Veriler ile icra edilen kod farklı bellek bölgesinde bulunur. Bu nedenle verileri işaret eden **veri adresleri kaydedicisi** (**data memory register**) işlemciye eklenmiştir.
- ◊ **Yığın bellek kaydedicisi** (**stack register**), fonksiyon çağrıları sırasında mevcut işlemci durumu ve yerel değişkenler gibi bazı verileri depolamak için kullanılan belleği gösteren (**stack pointer**) adresi tutan kaydedici işlemciye eklenmiştir.
- ◊ **Bayrak kaydedicisi** (**flag register**), işlemcinin durumu veya sıfır (**zero**) bayrağı, taşıma (**carry**) bayrağı, işaret (**sign**) bayrağı, taşma (**overflow**) bayrağı, eşlik (**parity**) bayrağı, yardımcı taşıma (**auxiliary carry**) bayrağı ve kesme etkinleştirme (**interrupt enable**) bayrağı gibi çeşitli işlemlerin sonucunu tutmak için kullanılan özel bir kaydedici işlemciye eklenmiştir.
- ◊ **Genel amaçla kullanılan kaydediciler** (**general purpose registers**) işlemciye eklenmiştir.

Eklenen bu kaydediciler ile doğal koda (**native code**) **derlenen her bir program**, dört bellek bölgesinden (**segment**) oluşur;

1. **Kod bellek** (**code segment**) ya da kaynak koda ithafla **metin bellek** (**text segment**) icra edilecek emirleri içeren bellektir.
2. Programın işleyeceği verileri tutan **veri belleği** (**data segment**).
3. Yığın bellek (**stack segment**), fonksiyon çağrıları sırasında kullanılan bellektir. Bu bellek, son giren veri ilk çıkar LIFO (**last in first out**) mantığıyla çalışır. Çünkü;
 - ◊ **Çağrıdan** önce mevcut işlemci durumu ve yerel değişkenler gibi bazı verileri depolamak ve
 - ◊ Fonksiyondan **dönülünce** işlemciyi ve çağrı ortamını eski hale getirmek için kullanılır.
4. Programın icrası sırasında dinamik olarak eklenip çıkarılan veriler için hurdalık gibi kullanılan **öbek bellek** (**heap segment**).

Klasik derlenmiş doğal kod (**native code**) dosyaları gibi, .NET altyapısında da ortak dil kodu (**CIL code**) da sanki sanal bir işlemci varmış gibi **bu dört bellek bölgesiyle çalışacak şekilde üretilir**. Çalıştırma ortamı (CLR) tarafından içinde bulunduğu işletim sistemine ait işlemcinin emir kodlarına JIT ile derlenir.

Ayrıca ortak dil tarifiğinde (CLS) belirtilen standartlara uymak zorunda olan birçok programlama dili bulunmaktadır; VB.NET, C#, VC++.NET, J#, F#, Jscript.NET, Windows PowerShell, Iron phyton, Iron Ruby. Bu dillerde yazılan kodlar derlenerek ortak ara dile (CIL) dönüştürülür.

3.6 .NET Framework Kurulumu

Çerçeve yapının son sürümü, resmi sitesinden indirilerek kurulur ¹. Kurulum tamamlandıktan sonra konsoldan aşağıdaki komut yazılarak kurulum teyit edilebilir;

```
Microsoft Windows [Version 10.0.22621.4317]
(c) Microsoft Corporation. Tüm hakları saklıdır.
C:\Users\ILHANOZKAN>dotnet
```

```
Usage: dotnet [options]
Usage: dotnet [path-to-application]
```

```
Options:
-h|--help          Display help.
--info             Display .NET information.
--list-sdks        Display the installed SDKs.
--list-runtimes    Display the installed runtimes.
```

```
path-to-application:
The path to an application .dll file to execute.
C:\Users\ILHANOZKAN>
```

Sonrasında **PATH** değişkenine C# Derleyicisinin yolu (C:\Windows\Microsoft.NET\Framework\v4.0.30319\) eklenir. Daha sonra kurulum, konsolda **csc -help** komutu yazılarak kontrol edilebilir. *Düzeltilir*

En sonunda metin editörü ile ilk kaynak kodumuzu oluşturmak için komut satırında **notepad hello.cs** komutu yazılır. Eğer böyle bir dosya yoksa editör bu dosyayı oluşturmak için onay ister. Açılan metin editörü penceresine aşağıdaki ilk C# programı yazılır ve **hello.cs** olarak dosyaya kaydedilir.

```
using System;
using System.IO;
namespace ConsoleApp1
{
    internal class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Hello, World!");
        }
    }
}
```

Bu kodu derlemek için **csc hello.cs** komutu konsolda yazılır. Artık .NET CIL kodumuzu barındıran **hello.exe** adlı **montaj (assembly)** uygulaması hazır. Derlenen programı çalıştırmak için yine aynı komut satırında **hello.exe** yazarak CLR tarafından icra edilebilir.

```
C:\Users\ILHANOZKAN>notepad hello.cs
```

```
C:\Users\ILHANOZKAN>csc hello.cs
Microsoft (R) Visual C# Compiler version 4.8.9232.0
for C# 5
Copyright (C) Microsoft Corporation. All rights reserved.
```

```
This compiler is provided as part of the Microsoft (R) .NET Framework,...
```

```
C:\Users\ilhan>hello.exe
Hello, World!
```

```
C:\Users\ILHANOZKAN>
```

Bazı durumlarda bir projede sadece sınıflar bulunur. Bu tip projelere sınıf kütüphanesi denir ve der-

¹<https://dotnet.microsoft.com/en-us/download>

lendiğinde *.dll dosyası oluşur. Bunlar derlenmiş olarak başka projelerde referans gösterilerek kullanılabilir. İleride örnekleri verilecektir.

3.7 Entegre Geliştirme Ortamı

Şimdiye kadar anlatılan kod yazma (**implementation**), derleme (**compile**), icra (**execute**) ya da koşma (**run**) ve izleme (**trace**) gibi süreçlerin tamamını tek bir çatı altında yürütmemizi sağlayan programlar, entegre geliştirme ortamı IDE (**Integrated Development Environment**) olarak adlandırılır. C# programlama dili için en çok kullanılan entegre geliştirme ortamları aşağıda verilmiştir;

- ◊ **Visual Studio**, Microsoft tarafından geliştirilen, C dilinde yazılmış, güçlü, yüksek performanslı uygulamalar oluşturmak için kullanılabilen bir geliştirme ortamıdır. Yalnızca Windows'ta çalışır. Visual Studio, kod tamamlama (**intellisense**), kullanıcı ara yüzü UI (**user interface**) hazırlama desteği ve hata ayıklama (**debugger**) ve birçok eklentisi (**plug-in**) olan muazzam özelliklere sahiptir. Bu geliştirme ortamının kurulumu dipnotta verilmiştir.
- ◊ **Visual Studio Code**, Microsoft tarafından geliştirilen açık kaynaklı bir geliştirme ortamıdır. Windows, macOS ve Linux gibi işletim sistemlerinde çalışır. Git sürüm kontrol (**version control**) sistemleriyle çok iyi çalışır. Ayrıca, akıllı kod tamamlamanın dikkate değer özellikleriyle birlikte gelir.
- ◊ **Xcode**, MacOSX işletim sisteminde yazılım geliştirmek için kullanılan bir entegre geliştirme ortamıdır.
- ◊ **JetBrains Rider**, Windows, Mac OS X ve Linux'ta .NET ve .NET Core uygulamalarını destekler. Rider'ın hızlı performansı, geniş platform ve çalışma zamanı desteği, sürüm kontrol sistemleriyle sorunsuz entegrasyonu ve kapsamlı derleme çözümleme (**decompile**) özellikleri öne çıkan özelliklerinden bazılarıdır.
- ◊ **MonoDevelop**, .NET dilleri için oluşturulmuş açık kaynaklı bir geliştirme ortamıdır. Yazılım geliştirme için Mono framework ve .NET framework'ü kullanır. MonoDevelop ile MacOSX, Windows ve Linux dahil olmak üzere çeşitli işletim sistemleri için masaüstü ve web uygulamalarını verimli bir şekilde oluşturabilirsiniz.
- ◊ **.NET Core CLI**, .NET Core uygulamalarını geliştirmek, oluşturmak, çalıştırmak ve yayınlamak için komut satırı arayüzüdür (**command line interface**). Bu arayüz adından da anlaşılacağı üzere grafik ara yüzünden yoksundur.
- ◊ Entegre geliştirme ortamları metin dosyası olarak tutulan kaynak kodları programcıya renklendirerek gösterir. Bu da programcının kodu anlamasını daha da kolaylaştırır. Ancak kodu dosyaya yine sade metin olarak kaydeder.

3.8 Çevrimiçi Derleyiciler

Bütün bu entegre geliştirme ortamlarının yanında çevrim içi olarak da derleme yapılan web uygulamaları da bulunmaktadır. Bunlardan birkaçı aşağıda verilmiştir;

- ◊ https://www.onlinegdb.com/online_csharp_compiler
- ◊ <https://www.tutorialspoint.com/csharp/index.htm>
- ◊ <https://onecompiler.com/csharp>

3.9 Derleme Zamanı

Derleyici programının derleme işlemine başlayıp bitirildiği sürece **derleme zamanı** (**compile-time**) denir. Bu süreç başarısızlıkla da sonuçlanabilir ve eğer derleme işleminde hata meydana gelirse programcı hata mesajları ile uyarılır.

Bir derleyici program, kaynak dosyayı makine diline çevirme çabasında, kaynak dosyanın C# dilinin söz dizimi kurallarına uygunluğunu da denetler. Eğer dilin kurallarına uyulmamışsa, derleyici bu durumu bildiren bir ileti de vermek zorundadır.

```
using System;
using System.IO;
namespace ConsoleApp1
{
    internal class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Hello, World!")
        }
    }
}
```

Yukarıda hatalı yazılmış bir C# programı derlendiğinde aşağıdaki hata alınacaktır.

```
C:\Users\ILHANOZKAN>csc.exe hello.cs
Microsoft (R) Visual C# Compiler version 4.8.9232.0
for C# 5
Copyright (C) Microsoft Corporation. All rights reserved.
```

This compiler is provided as part of the Microsoft (R) .NET Framework,...

```
hello.cs(10,47): error CS1002: ; bekleniyor
```

```
C:\Users\ILHANOZKAN>
```

3.10 Hatalar

Programcı tarafından kodlama yapılırken genellikle aşağıdaki üç tip hata yapılır;

1. **Derleme zamanı hataları (compile-time error)**: Bu tip hatalar genelde kullanılan dilin yazım kurallarına (**syntax**) uyulmadığından, talimatların (**statement**) yanlış yazılmasından ya da programcının kod metninde uygun olmayan karakterlerin kullanılmasından kaynaklanır.
2. **Çalışma zamanı hataları (run-time error)**: Kaynak kod, kurallara uygun olarak yazılmıştır ve herhangi bir yazım hatası bulunmaz. Bu tip hatalara en iyi örneklerden birisi sıfıra bölme hatasıdır. Taşma hataları da bu hatalardandır. Daha sonra değinilecektir.
3. **Mantıksal hatalar (logical error)**: Programcının çözüm için gerekli adımların oluşturulmasında, çözüm yönteminin yanlış olmasından ya da yanlış işleçler (**operator**) kullanmasından kaynaklanır. Örneğin bir büyüktür (>) işleci yerine küçüktür (<) işleci kullanıldığında ne bir yazım hatası ne de bir çalışma zamanı hatası ortaya çıkar. Fakat program kendisinden istenilen davranışları yerine getirmez ve uygun çıktıları üretmez. Faktöriyel hesaplanırken $0! = 1$ yerine $0! = 0$ kabul etmek buna örnek verilebilir.

3.11 C# Projesi Oluşturma ve Çalıştırma

§3.11.1 .NET Core CLI ile Proje Oluşturma

Konsolda komut satırı kullanılarak bir proje oluşturmak için aşağıdaki adımlar takip edilir:

1. Çalıştığımız herhangi bir klasörde yeni proje için konsolda `mkdir konsolprojesi` komutu ile yeni klasör oluşturulur.
2. Ardından oluşturulan klasöre girilir (`cd konsolprojesi`).
3. Yeni bir proje oluşturmak için `dotnet -new console` komutu çalıştırılır.
4. İki adet dosya oluşacaktır: Biri proje dosyası (`konsolprojesi.csproj`), diğeri ise `Program.cs` dosyasıdır.
5. Bir de projeye ilişkin amaç dosyaların olacağı `obj` adlı bir klasör de oluşur.

Oluşan Projenin Klasör Yapısı Aşağıda verilmiştir;

```
konsolprojesi/
├── Program.cs
├── konsolprojesi.csproj
├── obj/
│   ├── konsolprojesi.csproj.nuget.dgspec.json
│   ├── konsolprojesi.csproj.nuget.g.props
│   ├── konsolprojesi.csproj.nuget.g.targets
│   ├── project.assets.json
│   └── project.nuget.cache
```

Ana programımız olan `Program.cs` dosyası da en basit haliyle aşağıdaki gibi oluşur;

```
// See https://aka.ms/new-console-template for more information
Console.WriteLine("Hello, World!");
```

Ayrıca XML (**Extensible Markup Language**) olarak oluşan proje dosyasının (`konsolprojesi.csproj`) içeriği aşağıdaki gibidir;

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net9.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
```

```
</PropertyGroup>
</Project>
```

Oluşan projede **ana programı çalıştırmak için** proje klasöründe **dotnet run** komutu çalıştırılır.

```
C:\Users\ILHANÖZKAN\konsolprojesi>dotnet run
Hello, World!
```

Çalışma sonrasında proje klasörü aşağıdaki şekilde değişir;

```
konsolprojesi/
├── Program.cs
├── konsolprojesi.csproj
├── obj/
│   ├── Debug/
│   │   └── net10.0/
│   │       └── ...
│   ├── konsolprojesi.csproj.nuget.dgspec.json
│   ├── konsolprojesi.csproj.nuget.g.props
│   ├── konsolprojesi.csproj.nuget.g.targets
│   ├── project.assets.json
│   └── project.nuget.cache
├── bin/
│   └── Debug/
│       └── net10.0/
│           ├── konsolprojesi.deps.json
│           ├── konsolprojesi.dll
│           ├── konsolprojesi.exe
│           ├── konsolprojesi.pdb
│           └── konsolprojesi.runtimeconfig.json
```

Aslında icra edilen **./bin/Debug/net10.0/konsolprojesi.exe** ve dolaylı olarak **konsolprojesi.dll** montaj dosyalarıdır. Geri kalan dosyalar çalışma ortamı ile ilgili ayarlardır.

Çalıştırma değil de sadece derleme yapılacak ise iki tip derleme yapılabilir;

- ◊ Hata ayıklama (**debug**) amacıyla yapılacak ise **dotnet build** komutu ile derlenir.
- ◊ Yayım (**release**) amacıyla yapılacak ise **dotnet build -c Release** komutu ile derleme yapılır.

Bu adımlar MacOSX ve Linux işletim sistemleri üzerinde .NET yüklü ise aynen çalışır.

§3.11.2 Visual Studio ile Proje Oluşturma

Windows işletim sisteminde çalışan Visual Studio, kod geliştiriciler için en çok tercih edilen geliştirme ortamlarından biridir. İngilizce dile sahip Visual Studio programında sırasıyla şu adımlar takip edilir;

1. Programı ilk defa çalıştırıyorsanız, karşılama sayfasından **Create a new project** seçiniz. Eğer program açık ise **File** menüsüne giderek **File→New Project** seçiniz.
2. Proje tipi olarak **Console Application** seçilir.
3. **Project Name**, **Location** ve **Solution Name** belirlenerek ilk proje oluşturulmaya başlanır.
4. Projeye oluşturma sihirbazının **Additional Information** sayfasında **Do not use top-level statements** kutucuğu varsayılan olarak *seçili değildir*. Bu durumda projemiz yeni proje şablonuyla oluşur.
5. **Solution Explorer** üzerinde **Program.cs** dosyası açılır.

C#10+ ile birlikte **yeni proje şablonuyla** oluşan uygulamalar için **Program.cs** dosyası aşağıdaki şekilde oluşur;

```
1 // See https://aka.ms/new-console-template for more information
2 Console.WriteLine("Hello, World!");
```

Yeni proje şablonu ile oluşturulan projelerde **Program.cs** kodunun başında birinci satırdaki açıklama her zaman bulunur. Eski proje şablonuyla proje oluşturulmak istenirse yukarıda anlatılan 3. adımdaki *Additional Information* sayfasında **Do not use top-level statements** kutucuğu *seçili bırakılır*. Bu durumda **Program.cs** alışık olduğumuz eski şekilde oluşur;

```
1 namespace ConsoleApp1
2 {
3     internal class Program
4     {
5         static void Main(string[] args)
6         {
```



```

7         Console.WriteLine("Hello, World!");
8     }
9 }
10 }

```

Burada ilk oluşturduğumuz yeni tip `Program.cs` dosyasında yazılan kodları, ikinci oluşan eski tip `Program.cs` dosyasındaki `static void Main(string[] args)` yönteminin içerisinde yazılıyor gibi kabul edebilirsiniz. Yeni proje şablonuyla yazılan değişkenlerin "yerel değişken" değil, aslında gizli bir `Main` yönteminin içindeki değişkenlerdir, bu yüzden ana programın parametrelerine `args` parametresine hala erişilebilir.

Yeni proje şablonuyla oluşturulan `Program.cs` dosyasına aşağıdaki yönergeler uygulamaya örtük olarak eklenir. Bu kütüphanelerdeki sınıfları projemizde doğrudan kullanabiliriz;

```

◇ using System;
◇ using System.IO;
◇ using System.Collections.Generic;
◇ using System.Linq;
◇ using System.Net.Http;
◇ using System.Threading;
◇ using System.Threading.Tasks;

```

Visual Studio menüsünde, **hata ayıklama** için **Debug**→**Start Debugging** tıklanır veya **F5** tuşuna basılarak hata ayıklama modunda program çalıştırılır. Ya da hata ayıklamadan derleme yapmak için **Debug**→**Start Without Debugging** tıklanarak veya **Ctrl + F5** tuşlarına basarak program doğrudan çalıştırılabilir.

Visual Studio ile oluşturulan projelerde de komut satırından oluşturulan projeler gibi aynı klasör yapısında dosyalar oluşur.

Kitap içerisinde oluşturulan projelerde aksi belirtilmedikçe C#6+ sonrası kullanılan **yeni tip proje şablonu kullanılacaktır!**

3.12 Düşün ve Çöz

- ◇ **Düşün:** .NET'in Ortak Ara Dili (CIL/MSIL) ile Java'nın bytecode'u arasındaki benzerlik ve farklar nelerdir? 'Write once, run anywhere' felsefesi her iki platformda nasıl hayata geçirilir?
- ◇ **Düşün:** JIT (**Just-In-Time**) derleme ile AOT (**Ahead-Of-Time**) derleme arasındaki temel fark nedir? .NET'in ReadyToRun ve NativeAOT özellikleri hangi kullanım senaryolarında tercih edilir?
- ◇ **Düşün:** Derleme zamanı hatası (**compile-time error**) ile çalışma zamanı hatası (**run-time error**) arasındaki fark nedir? Hangisi daha tehlikelidir ve neden? Bir örnek üzerinden açıklayınız.
- ◇ **Düşün:** Ortak Dil Tarifnamesi (CLS) ve Ortak Tip Sistemi (CTS) olmadan C# ile F# veya VB.NET'in bir arada çalışması neden mümkün olmaz? Dil bağımsızlığının kurumsal yazılım projelerindeki önemi nedir?
- ◇ **Düşün:** Çöp toplayıcı GC (**garbage collector**) bellek yönetimini nasıl yapar? Çöp toplayıcının olmadığı bir ortamda (örneğin C/C++) bellek sızıntısına (**memory leak**) nasıl yol açılır? C#ta yine de bellek sızıntısı oluşabilir mi?
- ◇ **Düşün:** Visual Studio ile VS Code arasında bir C# projesi geliştirirken hangisini tercih edersiniz ve neden? Ticari bir proje için IDE seçimini etkileyen faktörler nelerdir?
- ◇ **Düşün:** .NET 8 ile .NET Framework 4.x arasındaki temel farklar nelerdir? Eski bir .NET Framework uygulamasını modern .NET'e taşımanın (migration) önünde hangi engeller bulunabilir?
- ◇ **Çöz:** C# kaynak kodunun .NET çalışma ortamında icra edilmesine kadar geçen adımları sırasıyla açıklayan bir akış diyagramı çiziniz: `.cs` dosyası→CIL→JIT→makine kodu→çalışma.
- ◇ **Çöz:** Komut satırından (CLI) yeni bir C# konsol uygulaması oluşturan, derleyen ve çalıştıran komutları adım adım yazınız. Her komutun ne yaptığını açıklayınız.
- ◇ **Çöz:** Aşağıdaki kodda kaç tür hata bulunabilir? Her hatayı sınıflandırınız (söz dizimi/mantık/çalışma zamanı): `int x = 10; int y = 0; Console.WriteLine(x / y);`
- ◇ **Çöz:** `using System;` direktifinin olmadığı bir projede `Console.WriteLine()` çağrısının neden derleme hatası verdiğini açıklayınız. Bunu çözmek için iki farklı yolu gösteriniz.
- ◇ **Çöz:** `dotnet run` ile `dotnet build + dotnet ./bin/.../app.dll` komutları arasındaki farkı açıklayınız. Hangi senaryoda hangisi tercih edilir?
- ◇ **Düşün:** C# kodu neden doğrudan işlemcinin anlayacağı makine koduna değil de önce bir 'Ara Dil' (CIL) koduna dönüştürülür? Bu durumun platform bağımsızlığıyla ilgisi nedir?

- ◇ **Düşün:** Visual Studio'da 'Hata Ayıklama' (**debug**) modu ile 'Hata Ayıklamadan Başlat' arasındaki temel fark, programın çalışma hızı ve geliştiriciye sunduğu bilgiler açısından nelerdir?
- ◇ **Düşün:** Yeni C# proje şablonlarında neden **static void Main** metodunu görmüyoruz? Arka planda derleyici bu kodu bizim yerimize nasıl oluşturuyor?